ROLL NO : 241801038

EX 6: Error Detection and Correction Using Hamming Code

AIM :

To write a Python program to implement error detection and correction using Hamming Code.

ALGORITHM :

1. Start the program.
2. Read the number of 4-bit data blocks.
3. Arrange the data bits in positions 3, 5, 6 and 7, and parity bits in positions 1, 2 and 4.
4. Calculate parity bits P1, P2 and P4 using even parity.
5. Generate the 7-bit Hamming code word.
6. Introduce a single-bit error at the position specified by the user.
7. Recalculate the check bits C1, C2 and C4 at the receiver.
8. Calculate the error position using the syndrome bits.
9. If an error is detected, invert the faulty bit to correct it. Otherwise, display that no error is detected.
10. Extract the original 4 data bits from the corrected code word.
11. Repeat the process for all data blocks.
12. Stop the program.

Code :

```
def encode(d):
    h = [0] * 7

    h[2] = d[0]
    h[4] = d[1]
```

```
    h[5] = d[2]
    h[6] = d[3]

    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]

    return h


def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]

    error_pos = c1 + c2 * 2 + c4 * 4

    if error_pos != 0:
        print("Error detected at bit position:", error_pos)
        h[error_pos - 1] ^= 1
        print("Error corrected.")
    else:
        print("No error detected.")

    return error_pos


def to_str(bits):
    return "".join(map(str, bits))


def main():
    n = int(input("Enter number of 4-bit data blocks: "))

    for b in range(1, n + 1):
        print("\n--- Block", b, "---")

        d = list(map(int, input(
            "Enter 4 data bits separated by spaces: "
        ).split()))

        h = encode(d)

        print("Original Data Bits:", to_str(d))
        print("Encoded Hamming Code:", to_str(h))

        pos = int(input(
            "Enter error position (1-7, 0 for no error): "
        ))

        if 1 <= pos <= 7:
```

```
                h[pos - 1] ^= 1

            print("Received Code:", to_str(h))

            detect_and_correct(h)

            print("Corrected Code:", to_str(h))

            recovered = [h[2], h[4], h[5], h[6]]
            print("Recovered Data Bits:", to_str(recovered))


    main()
```

```
Enter number of 4-bit data blocks: 2

--- Block 1 ---
Enter 4 data bits separated by spaces: 1 0 1 1
Original Data Bits: 1011
Encoded Hamming Code: 0110011
Enter error position (1-7, 0 for no error): 2
Received Code: 0010011
Error detected at bit position: 2
Error corrected.
Corrected Code: 0110011
Recovered Data Bits: 1011

--- Block 2 ---
Enter 4 data bits separated by spaces: 0 0 1 1
Original Data Bits: 0011
Encoded Hamming Code: 1000011
Enter error position (1-7, 0 for no error): 5
Received Code: 1000111
Error detected at bit position: 5
Error corrected.
Corrected Code: 1000011
Recovered Data Bits: 0011
```

Result :

The Python program for error detection and correction using Hamming Code was successfully executed. The single-bit errors were detected and corrected, and the original data bits were recovered successfully.